Binary Search Tree / File Manager

Generated by Doxygen 1.18.0

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 BST_System Class Reference . . . 5

3.1.1 Detailed Description . . . 5

3.1.2 Constructor & Destructor Documentation . . . 5

3.1.2.1 BST_System() . . . 5

3.1.3 Member Function Documentation . . . 6

3.1.3.1 CreateNode() . . . 6

3.1.3.2 DeleteNode() . . . 6

3.1.3.3 EraseNode() . . . 6

3.1.3.4 FindNode() . . . 6

3.1.3.5 InorderTraversalNode() . . . 6

3.1.3.6 InsertNode() . . . 6

3.1.3.7 minValueNode() . . . 7

3.1.3.8 PostorderTraversalNode() . . . 7

3.1.3.9 PreorderTraversalNode() . . . 7

3.2 FileManager_System Class Reference . . . 7

3.2.1 Detailed Description . . . 7

3.2.2 Member Function Documentation . . . 8

3.2.2.1 LoadFromBinary() . . . 8

3.2.2.2 SaveToBinary() . . . 8

3.3 Node Struct Reference . . . 8

3.3.1 Detailed Description . . . 8

3.3.2 Constructor & Destructor Documentation . . . 9

3.3.2.1 Node() [1/2] . . . 9

3.3.2.2 Node() [2/2] . . . 9

3.3.3 Member Data Documentation . . . 9

3.3.3.1 data . . . 9

3.3.3.2 L . . . 9

3.3.3.3 R . . . 9

4 File Documentation 11

4.1 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/binary_tree.cpp File Reference . . . 11

4.2 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/binary_tree.cpp . . . 11

4.3 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/binary_tree.h File Reference . . . 13

4.3.1 Detailed Description . . . 13

4.4 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/binary_tree.h 13

4.5 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/file_manager.cpp File Reference 14

4.6 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/file_manager.cpp 14

4.7 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/file_manager.h File Reference 15

4.8 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/file_manager.h 15

4.9 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/main.cpp File Reference 15

4.9.1 Function Documentation 16

4.9.1.1 main() 16

4.10 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/main.cpp 16

Index **19**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BST_System 5
FileManager_System 7
Node
This is a class and struct to control and dictate how a Binary Search Tree would work 8

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/ **binary_tree.cpp** 11
C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/ **binary_tree.h**
Plik nagłówkowy algorytmu drzewa wyszukiwania binarnego, który został zaimplementowany w C++ 13
C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/ **file_manager.cpp** 14
C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/ **file_manager.h**
Plik nagłówkowy klasy menedżera plików umożliwiający prosty zapis/odczyt plików binarnych . 15
C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/ **main.cpp** 15

Chapter 3

Class Documentation

3.1 BST_System Class Reference

`#include <binary_tree.h>`

Public Member Functions

- **BST_System** (int o)
- **Node** ∗ **CreateNode** (int d) const
- **Node** ∗ **InsertNode** (**Node** ∗root, int d)
- **Node** ∗ **DeleteNode** (**Node** ∗root, int d)
- **Node** ∗ **FindNode** (**Node** ∗root, int key)

Static Public Member Functions

- static **Node** ∗ **minValueNode** (**Node** ∗node)
- static void **PreorderTraversalNode** (**Node** ∗root, int depth)
- static void **InorderTraversalNode** (**Node** ∗root, int depth)
- static void **PostorderTraversalNode** (**Node** ∗root, int depth)
- static void **EraseNode** (**Node** ∗root)

3.1.1 Detailed Description

Definition at line **24** of file **binary_tree.h**.

3.1.2 Constructor & Destructor Documentation

3.1.2.1 BST_System()

```
BST_System::BST_System (
            int o)  [inline]
```

Definition at line **28** of file **binary_tree.h**.

3.1.3 Member Function Documentation

3.1.3.1 CreateNode()

```
Node * BST_System::CreateNode (
            int d) const
```

Definition at line **7** of file **binary_tree.cpp**.

3.1.3.2 DeleteNode()

```
Node * BST_System::DeleteNode (
             Node * root,
            int d)
```

Definition at line **21** of file **binary_tree.cpp**.

3.1.3.3 EraseNode()

```
void BST_System::EraseNode (
             Node * root)  [static]
```

Definition at line **115** of file **binary_tree.cpp**.

3.1.3.4 FindNode()

```
Node * BST_System::FindNode (
             Node * root,
            int key)
```

Definition at line **94** of file **binary_tree.cpp**.

3.1.3.5 InorderTraversalNode()

```
void BST_System::InorderTraversalNode (
             Node * root,
            int depth)  [static]
```

Definition at line **65** of file **binary_tree.cpp**.

3.1.3.6 InsertNode()

```
Node * BST_System::InsertNode (
             Node * root,
            int d)
```

Definition at line **14** of file **binary_tree.cpp**.

3.1.3.7 minValueNode()

```
Node * BST_System::minValueNode (
            Node * node)  [static]
```

Definition at line **109** of file **binary_tree.cpp**.

3.1.3.8 PostorderTraversalNode()

```
void BST_System::PostorderTraversalNode (
            Node * root,
           int depth)  [static]
```

Definition at line **74** of file **binary_tree.cpp**.

3.1.3.9 PreorderTraversalNode()

```
void BST_System::PreorderTraversalNode (
            Node * root,
           int depth)  [static]
```

Definition at line **56** of file **binary_tree.cpp**.

The documentation for this class was generated from the following files:

- C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/ **binary_tree.h**
- C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/ **binary_tree.cpp**

3.2 FileManager_System Class Reference

`#include <file_manager.h>`

Static Public Member Functions

- static void **SaveToBinary** (**Node** *root, const std::string &filename)
- static **Node** * **LoadFromBinary** (**Node** *root, const std::string &filename, **BST_System** &bst)

3.2.1 Detailed Description

Definition at line **8** of file **file_manager.h**.

3.2.2 Member Function Documentation

3.2.2.1 LoadFromBinary()

```
Node * FileManager_System::LoadFromBinary (
            Node * root,
           const std::string & filename,
            BST_System & bst)  [static]
```

Definition at line **22** of file **file_manager.cpp**.

3.2.2.2 SaveToBinary()

```
void FileManager_System::SaveToBinary (
            Node * root,
           const std::string & filename)  [static]
```

Definition at line **11** of file **file_manager.cpp**.

The documentation for this class was generated from the following files:

- C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/ **file_manager.h**
- C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/ **file_manager.cpp**

3.3 Node Struct Reference

This is a class and struct to control and dictate how a Binary Search Tree would work.

`#include <binary_tree.h>`

Public Member Functions

- **Node** (int o)
- **Node** ()

Public Attributes

- int **data**
- **Node** * **L**
- **Node** * **R**

3.3.1 Detailed Description

This firstly introduces the **Node** (p. 8), with data and then left and right as would the real binary tree. BTS is just a root with the left having smaller than and the right having bigger than numerals. We can

Definition at line **18** of file **binary_tree.h**.

3.3.2 Constructor & Destructor Documentation

3.3.2.1 Node() [1/2]

```
Node::Node (
            int o)  [inline]
```

Definition at line **20** of file **binary_tree.h**.

3.3.2.2 Node() [2/2]

```
Node::Node ()  [inline]
```

Definition at line **21** of file **binary_tree.h**.

3.3.3 Member Data Documentation

3.3.3.1 data

```
int Node::data
```

Definition at line **19** of file **binary_tree.h**.

3.3.3.2 L

```
Node* Node::L
```

Definition at line **19** of file **binary_tree.h**.

3.3.3.3 R

```
Node* Node::R
```

Definition at line **19** of file **binary_tree.h**.

The documentation for this struct was generated from the following file:

- C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/ **binary_tree.h**

Chapter 4

File Documentation

4.1 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/binary_tree.cpp File Reference

```
#include <vector>
#include <string>
#include <iostream>
#include "binary_tree.h"
```

4.2 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/binary_tree.cpp

Go to the documentation of this file.

```
00001 #include <vector>
00002 #include <string>
00003 #include <iostream>
00004 using namespace std;
00005 #include "binary_tree.h"
00006
00007 Node* BST_System::CreateNode(int d) const {
00008      Node* NewNode = new Node();
00009      NewNode->data = d; NewNode->L = NewNode->R = nullptr;
00010      return NewNode;
00011      cout « "New Node created:" « d « "\n";
00012 }
00013
00014 Node* BST_System::InsertNode(Node *root, int data){
00015     if (root == nullptr) return CreateNode(data);
00016     if (data < root->data) root->L = InsertNode(root->L, data);
00017     else if (data > root->data) root->R = InsertNode(root->R, data);
00018     return root;
00019 }
00020
00021 Node* BST_System::DeleteNode(Node* root,int d) {
00022     if (root == nullptr) return root;
00023
00024     // If the data to be deleted is smaller than the root's data, then it lies in the left subtree
00025     if (d < root->data) root->L = DeleteNode(root->L, d);
00026
00027     // If the data to be deleted is greater than the root's data, then it lies in the right subtree
00028     else if (d > root->data) root->R = DeleteNode(root->R, d);
00029
```

```
00030     // if data is same as root's data, then this is the node to be deleted
00031     else {
00032         // node with only one child or no child
00033         if (root->L == nullptr) {
00034             Node* temp = root->R;
00035             delete root;
00036             return temp;
00037         }
00038         if (root->R == nullptr) {
00039             Node* temp = root->L;
00040             delete root;
00041             return temp;
00042         }
00043         // node with two children : Get the inorder successor
00044         // (smallest in the right subtree)
00045         Node* temp = minValueNode(root->R);
00046 
00047         // Copy the inorder successor's content to this node
00048         root->data = temp->data;
00049 
00050         // Delete the inorder successor
00051         root->R = DeleteNode(root->R, temp->data);
00052     }
00053     return root;
00054 }
00055 
00056 void BST_System::PreorderTraversalNode(Node* root, int depth) {
00057     if (root != nullptr) {
00058         for (int i = 0; i < depth; i++) std::cout << (i == depth - 1 ? "|-- " : "    ");
00059         std::cout << root->data << "\n";
00060         PreorderTraversalNode(root->L, depth + 1);
00061         PreorderTraversalNode(root->R, depth + 1);
00062     }
00063 }
00064 
00065 void BST_System::InorderTraversalNode(Node* root, int depth) {
00066     if (root != nullptr) {
00067         InorderTraversalNode(root->R, depth + 1);
00068         for (int i = 0; i < depth; i++) std::cout << "    ";
00069         std::cout << "--- " << root->data << "\n";
00070         InorderTraversalNode(root->L, depth + 1);
00071     }
00072 }
00073 
00074 void BST_System::PostorderTraversalNode(Node* root, int depth) {
00075     if (root != nullptr) {
00076         PostorderTraversalNode(root->L, depth + 1);
00077         PostorderTraversalNode(root->R, depth + 1);
00078         for (int i = 0; i < depth; i++) std::cout << "    ";
00079         std::cout << "-> " << root->data << "\n";
00080     }
00081 }
00082 
00083 bool BST_System::FindPathRec(Node* node, int key, std::vector<int>& path) {
00084     if (node == nullptr) return false;
00085 
00086     path.push_back(node->data);
00087 
00088     if (node->data == key) return true;
00089     if (key < node->data) return FindPathRec(node->L, key, path);
00090 
00091     return FindPathRec(node->R, key, path);
00092 }
00093 
00094 Node *BST_System::FindNode(Node *root, int key) {
00095     std::vector<int> path;
00096 
00097     if (FindPathRec(root, key, path)) {
00098         std::cout << "Droga do waluty " << key << ": ";
00099         for (size_t i = 0; i < path.size(); i++) {
00100             std::cout << path[i];
00101             if (i != path.size() - 1) std::cout << " -> ";
00102         }
00103         std::cout << "\n";
00104     } else {
00105         std::cout << "Waluta " << key << " nie byla znaleziona.\n";
00106     }
00107 }
00108 
00109 Node* BST_System::minValueNode(Node* node){
00110     Node* current = node;
00111     while (current && current->L != nullptr) current = current->L;
00112     return current;
00113 }
00114 
00115 void BST_System::EraseNode(Node* node) {
00116     if (node != nullptr) {
```

```
00117         EraseNode(node->L);
00118         EraseNode(node->R);
00119         delete node;
00120     }
00121 }
```

4.3 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/binary_tree.h File Reference

Plik nagłówkowy algorytmu drzewa wyszukiwania binarnego, który został zaimplementowany w C++.

```
#include <vector>
#include <string>
#include <iostream>
```

Classes

- struct **Node**

  This is a class and struct to control and dictate how a Binary Search Tree would work.
- class **BST_System**

4.3.1 Detailed Description

Author

Kacper Twaróg

Date

2026-11-12

Definition in file **binary_tree.h**.

4.4 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/binary_tree.h

Go to the documentation of this file.

```
00001 #ifndef SOURCE_CODE_BINARY_TREE_H
00002 #define SOURCE_CODE_BINARY_TREE_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <iostream>
00007 using namespace std;
00008
00017
00018 struct Node {
00019      int data; Node* L; Node* R;
00020      Node(int o) : data(o), L(nullptr), R(nullptr) {}
```

```
00021      Node() : data(0), L(nullptr), R(nullptr) {}
00022 };
00023
00024 class BST_System {
00025     int data; Node* L; Node* R;
00026     static bool FindPathRec(Node* node, int key, std::vector<int>& path);
00027 public:
00028     BST_System(int o) : data(o), L(nullptr), R(nullptr) {}
00029     Node* CreateNode(int d) const;
00030     Node* InsertNode(Node *root, int d);
00031     Node* DeleteNode(Node *root, int d);
00032
00033     Node *FindNode(Node *root, int key);
00034     static Node* minValueNode(Node* node);
00035     static void PreorderTraversalNode(Node *root, int depth);
00036     static void InorderTraversalNode(Node *root, int depth);
00037     static void PostorderTraversalNode(Node *root, int depth);
00038     static void EraseNode(Node* root);
00039 };
00040
00041 #endif
```

4.5 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/file_manager.cpp File Reference

```
#include "file_manager.h"
```

4.6 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/file_manager.cpp

Go to the documentation of this file.

```
00001 #include "file_manager.h"
00002
00003 void FileManager_System::SaveBinaryHelper(Node* root, std::ofstream& file) {
00004     if (root != nullptr) {
00005         file.write(reinterpret_cast<const char*>(&(root->data)), sizeof(int));
00006         SaveBinaryHelper(root->L, file);
00007         SaveBinaryHelper(root->R, file);
00008     }
00009 }
00010
00011 void FileManager_System::SaveToBinary(Node* root, const std::string& filename) {
00012     std::ofstream file(filename, std::ios::binary);
00013     if (file.is_open()) {
00014         SaveBinaryHelper(root, file);
00015         file.close();
00016         std::cout « "Tree saved to binary file: " « filename « "\n";
00017     } else {
00018         std::cout « "Error opening binary file for writing.\n";
00019     }
00020 }
00021
00022 Node* FileManager_System::LoadFromBinary(Node* root, const std::string& filename, BST_System& bst) {
00023     std::ifstream file(filename, std::ios::binary);
00024     if (!file.is_open()) {
00025         std::cout « "Error opening binary file for reading.\n"; return root;
00026     }
00027     int val;
00028     while (file.read(reinterpret_cast<char*>(&val), sizeof(int))) root = bst.InsertNode(root, val);
00029     file.close();
00030     std::cout « "Data loaded from binary file.\n";
00031     return root;
00032 }
```

4.7 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/file_manager.h File Reference

Plik nagłówkowy klasy menedżera plików umożliwiający prosty zapis/odczyt plików binarnych.

```
#include "binary_tree.h"
#include <fstream>
#include <iostream>
#include <string>
```

Classes

- class **FileManager_System**

4.8 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/file_manager.h

Go to the documentation of this file.

```
00001 #ifndef SOURCE_CODE_FILE_MANAGER_H
00002 #define SOURCE_CODE_FILE_MANAGER_H
00003 #include "binary_tree.h"
00004 #include <fstream>
00005 #include <iostream>
00006 #include <string>
00007
00008 class FileManager_System {
00009     static void SaveBinaryHelper(Node* root, std::ofstream& file);
00010
00011 public:
00012     static void SaveToBinary(Node* root, const std::string& filename);
00013     static Node* LoadFromBinary(Node* root, const std::string& filename, BST_System& bst);
00014 };
00015
00016 #endif
```

4.9 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/main.cpp File Reference

```
#include <iostream>
#include "binary_tree.h"
#include "file_manager.h"
```

Functions

- int **main** ()

4.9.1 Function Documentation

4.9.1.1 main()

```
int main ()
```

Definiowanie różnych zmiennych do obsługi systemu.

Parameters

a	Tymczasowa zmienna rozpoczynająca drzewo binarne, ponieważ regularnie zaczynała się od 0, przerwała sekwencjonowanie, ponieważ korzeń stał się 0, dlatego wszystkie liczby byłyby po jednej stronie.
BST	Instancja klasy Binary Search Tree.
FM	Instancja klasy Menedżer plików odpowiedzialna za konwersję i odszyfrowanie z czystego pliku binarnego.
root	Korzeń naszego drzewa binarnego i na czym opiera się cały ten program.
filen	Reprezentuje nazwę pliku, na której będziemy polegać podczas pracy z operacjami wejścia i wyjścia systemu.
choice	Prosty int pozwala nam używać przypadków przełączników do przechodzenia przez nieskończoną liczbę wyborów symulujących menu w celu przetestowania modułu drzewa wyszukiwania binarnego.

Definition at line **20** of file **main.cpp**.

4.10 C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/main.cpp

Go to the documentation of this file.

```
00001 #include <iostream>
00002 #include "binary_tree.h"
00003 #include "file_manager.h"
00004
00009
00014
00019
00020 int main() {
00021     int a; std::cout « "Wybierz liczbe jako srodek drzewa binarnego: ";
00022     std::cin » a;
00023     BST_System BST = a;
00024     FileManager_System FM;
00025     Node* root = BST.CreateNode(a);
00026     string filen;
00027     int choice = -1;
00028
00040
00041     while (choice != 0) {
00042         std::cout « "\n--- MENU ---\n";
00043         std::cout « "67. Dodaj przykladowe drzewo\n";
00044         std::cout « "1. Dodaj Element\n";
00045         std::cout « "2. Usuń Element\n";
00046         std::cout « "3. Usuń Całe Drzewo\n";
00047         std::cout « "4. Pokaż Drzewo (Pre-Order)\n";
00048         std::cout « "5. Pokaż Drzewo (In-Order)\n";
00049         std::cout « "6. Pokaż Drzewo (Post-Order)\n";
00050         std::cout « "7. Znajdz droge do Elementu\n";
00051         std::cout « "8. Zapisz do pliku\n";
00052         std::cout « "9. Wczytaj z pliku (overwrite)\n";
00053         std::cout « "11. Wczytaj z pliku (append)\n";
00054         std::cout « "0. Wyjdz\n";
00055         std::cout « "Wybierz opcje: ";
00056         std::cin » choice;
00057
```

```
00058         switch (choice) {
00059             case 67:
00060                 BST.InsertNode(root, 50);
00061                 BST.InsertNode(root, 60);
00062                 BST.InsertNode(root, 25);
00063                 BST.InsertNode(root, 3);
00064                 BST.InsertNode(root, 150);
00065                 BST.InsertNode(root, 70);
00066                 break;
00067             case 1: {
00068                 int v; std::cout « "Dodaj walute do drzewa: ";
00069                 std::cin » v; root = BST.InsertNode(root, v);
00070                 break;
00071             }
00072             case 2: {
00073                 int v; std::cout « "Usuń walute z drzewa: ";
00074                 std::cin » v; root = BST.DeleteNode(root, v);
00075                 break;
00076             }
00077             case 3:
00078                 BST.EraseNode(root);
00079                 std::cout « "Drzewo usunięte.\n";
00080                 break;
00081             case 4:
00082                 BST.PreorderTraversalNode(root, 0); break;
00083             case 5:
00084                 BST.InorderTraversalNode(root, 0); break;
00085             case 6:
00086                 BST.PostorderTraversalNode(root, 0); break;
00087             case 7:
00088                 int v; std::cout « "Wybierz walute do wyszukania: ";
00089                 std::cin » v; root = BST.FindNode(root, v); break;
00090             case 8:
00091                 std::cout « "Podaj imie pliku do ktorego zapisac: ";
00092                 std::cin » filen; FM.SaveToBinary(root, filen); break;
00093             case 9: {
00094                 std::cout « "Podaj imie pliku do wczytania (overwrite): ";
00095                 root = nullptr;
00096                 std::cin » filen; root = FM.LoadFromBinary(root, filen, BST);
00097                 break; }
00098             case 11: {
00099                 std::cout « "Podaj imie pliku do wczytania (append): ";
00100                 std::cin » filen; root = FM.LoadFromBinary(root, filen, BST);
00101                 break;
00102             }
00103             case 0:
00104                 BST.EraseNode(root);
00105                 std::cout « "Wychodzenie z programu...";
00106             default:
00107                 break;
00108         }
00109     }
00110     return 0;
00111 }
```

Index

BST_System, 5
    BST_System, 5
    CreateNode, 6
    DeleteNode, 6
    EraseNode, 6
    FindNode, 6
    InorderTraversalNode, 6
    InsertNode, 6
    minValueNode, 6
    PostorderTraversalNode, 7
    PreorderTraversalNode, 7

C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/binary_tree.cpp, 11
C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/binary_tree.h, 13
C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/file_manager.cpp, 14
C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/file_manager.h, 15
C:/Users/ripja/Documents/[Resources] GitHub Stuff/Advanced Programming C Projects/PRJ 2 - BST Tree Manager/Source Code/main.cpp, 15
CreateNode
    BST_System, 6

data
    Node, 9
DeleteNode
    BST_System, 6

EraseNode
    BST_System, 6

FileManager_System, 7
    LoadFromBinary, 8
    SaveToBinary, 8
FindNode
    BST_System, 6

InorderTraversalNode
    BST_System, 6
InsertNode
    BST_System, 6

L
    Node, 9

LoadFromBinary
    FileManager_System, 8

main
    main.cpp, 16
main.cpp
    main, 16
minValueNode
    BST_System, 6

Node, 8
    data, 9
    L, 9
    Node, 9
    R, 9

PostorderTraversalNode
    BST_System, 7
PreorderTraversalNode
    BST_System, 7

R
    Node, 9

SaveToBinary
    FileManager_System, 8